— *Solution notes* —

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 5

7 Concurrent and Distributed Systems (djg11)

State machines.

(*a*) Explain what happens to the state space, the possible behaviours and the reachable state space when two automata are coupled. [3 marks]

Answer: The state space of the combined machine is the Cartesian product of the individual state spaces. The reachable state space is defined with respect to a reset or starting state or set of states. The reachable state space is the subset of states that can be entered under any pattern of external inputs starting from a start state. The possible behaviours are reduced as machines are coupled (either synchronously or asynchronously) since some of the inputs have restricted values. The reachable state space is potentially reduced along with the behaviour, but it could be just a matter of deleting transitions with the state space being unaffected.

Full credit for any three, distinct, valid statements. (For example, in digital logic, an FSM may not be able to reach all of the potential states representable by its flip-flops, since it is a composition of the individual machines represented by each flip-flop).

Banker's Algorithm

(*b*) The Banker's Algorithm can be viewed as a predicate over shared state. What state does it operate over and does this include the program counter of the participating threads? When does it return true or false? [6 marks]

Answer: The Banker's Algorithm relies on a previous (automatic or manual) static analysis of each component machine or thread to determine the maximum number of each type of shared resource that it will need before that component halts/finishes.

Generally, the 'state' of a thread is highly characterised by its PC and control-flow graph, but the Banker's Algorithm is oblivious to this, beyond the static analysis phase.

The state vector examined by the algorithm is the number of resources of each type allocated to each component and the number of free resources of each type. (Not asked for in the question: The algorithm is run on a tentatively modified vector of the real state at each allocation request to determine whether to allow the allocation.)

The algorithm returns true (say) if the state is 'safe' meaning that there is a path forward where all components can finish without deadlock. (Not asked for in the question: The algorithm simulates forward to find at least one feasible interleaving.)

Isolation

(*c*) What is the difference between strict and non-strict isolation in a transaction processing system? What do both approaches ensure? Which can lead to a transaction abort being forced by the system and why? [3 marks]

Answer: In a set of overlapping transactions that operate on shared data, strict isolation prevents any transaction operating on (reading or writing) a datum operated on by another, except overlaps of read-only transactions with other read-only transactions can be ignored.

Both approaches ensure the C and I properties of ACID and that the overall effect is serialisable. (Strict isolation does not help ensure the A property. Atomicity (in the ACID sense of the word) means there are no partially committed transactions in the case of a crash, whereas isolation is about concurrency. In the context of multithreading, the word 'atomic' is about concurrency, whereas in the context of ACID it's about crash consistency.)

In non-strict isolation, the transactions are allowed to operate on overlapping data, but mechanisms are required to maintain serialisibilty.

Both strict and non-strict isolation can lead to an abort being forced by the system: in the non-strict case, an abort happens if a serialisability violation is detected; in the strict case, it happens if a deadlock is detected. Strict isolation is more likely to encounter deadlocks since it involves more waiting for locks.

(Serialisabiliy is violated when non-strict isolation has allowed an interleaving pattern that is invalid: e.g. it has read data that has been subsequently committed in a modified form, or e.g. it has read data that was modified by a transaction that itself was undone through a voluntary or enforced abort.)

Isolation

(*d*) "Increment and decrement operations are freely commutable" — what two assumptions are required for this statement to hold? Is it true that the effects of transactions containing increment and decrement operations are always serialisable? [3 marks]

Answer: If the increment and decrement operations are themselves atomic and the values they operate on have unbounded range, then they are comutable.

If the transactions *only* contain increment and decrement operations, their effects will be serialisable under any interleaving, but this is not the case if they otherwise read or write the shared data.

Atomic operations.

(*e*) Customers interact with a transaction processing system over a web interface but confirmations are also sent by email, such as 'please collect from your local branch'.

Should the email message be generated before, during, or after the process of committing the order transaction? What are the advantages and disadvantages of different approaches? Fully justify at least two design decisions in terms of system complexity and durability over a system crash.

[5 marks]

Answer: A major design point is not to send the email as a side effect of one of the transaction's constituent operations: instead it must be sent at or after the transaction is committed. Further, sending it as part of the commit adds complexity to a critical aspect of the transaction processing system. Instead, committing an entry to an 'emails to be sent' queue structure is preferable. This does not add any email-specific complexity to the transaction processing system.

Sending the email as part of the commit adds complexity to a critical aspect of the transaction processing system. If the email-sending infrastructure should be temporarily unavailable for some reason, it is better to commit the order transaction first and send the confirmation email later. After all, the email protocol is asynchronous anyway; email typically passes through multiple servers, and it may be queued and delayed at any of the servers it passes through. However, a delayed confirmation email could lead users to potentially submitting a duplicate order in the belief that the first order was not processed, so it might be necessary to detect and rollback/refund such duplicate orders.

A companion application or daemon should asynchronously serve the queue structure to de-queue the email requests and send them out. Another key point is this companion cannot be perfect owing to lack of atomicity: it has to operate on two independent entities (email and database) with neither being definitive.

Recovery from a system crash sometimes requires transaction rollback. Rollback of a sent email could involve sending a cancellation email, but this preferably should be avoided. In our case, email is only ever schedulled for committed transactions. If the system crashes before the commit is written out to persistent store, recovery involves re-doing the committed transactions, not undoing them, so no rollback of email's queued to be sent is required. Hence there is no problem here.

Also, owing to write ahead logging, or similar, there is no problem of an email being sent despite the transaction commit not being written out to persistent store, even if the daemon sends the message within a checkpoint epoch. *A poor answer might suggest that delaying the send until after the checkpoint is the solution, but this would just be a conservative auxiliary technique.*

A minor, unsolvable problem still arises for the companion daemon. The daemon will be coded as robustly as it can with respect to committing the 'successfully sent' status to the reliable structure at the same time as sending the email (e.g. based on the SMTP acknowledgement message etc), but if a crash happens between it getting the acknowledgement and the database update being entered into the write-ahead log there is a risk of the email being sent twice. But this is harmless enough.

Given open-book nature of exams, this sort of question should be preferred? Details of email protocols are not expected. Full marks for any 5 well-argued points.
